Liliana Berenguer Jin
25353882

Weather Planner

Internet Applications | Assignment 1 **Word count: 1518 words**

1. Introduction And Requirements

"Weather Planner" is a small reactive Internet Application that takes a destination city and tells the traveller what to pack given its weather forecast. The client runs in the browser using **Vue 3**. It talks to a **Node.js / Express** server, which does the actual work: it calls **OpenWeather's APIs**[1], reduces what comes back into a short summary, and returns that as one JSON object for the client to display.

Before writing any code, I listed the core requirements from the assignment to match each one to the OpenWeather API that could provide the data. In retrospect, this was a really useful starting point for deciding how to structure the **server side** code. Table 1 shows more or less what I concluded from it:

Requirement	OpenWeather API	What it returns
Turn the city name into a location	Geocoding (`/geo/1.0/direct`)	Latitude and longitude of city
Three-day table of temperature, wind speed and rainfall	5-day / 3-hour Forecast (`/data/2.5/forecast`)	~40 readings at 3-hour intervals
Umbrella if rain is forecast + Cold / Mild / Hot packing advice (below 8 °C, 8–24 °C, above 24 °C)	Forecast (same as above)	Rain and temperature per reading
Warning for each pollutant above the "Good" level, with its level and health risk	Air Pollution (`data/2.5/air_pollution`)	AQI (1–5) and the concentration of each pollutant
Compare two cities side by side (extra feature)	All of the above	-

Table 1: Requirements and the OpenWeather API used for each

The APIs themselves also set the order of the requests to OpenWeather clear: the **Forecast API** and **Air Pollution APIs** take coordinates rather than a city name, so Geocoding always has to go first.

[1] All of OpenWeather's API documentation: https://openweathermap.org/api/one-call-4?collection=one_call_api

2. System Architecture

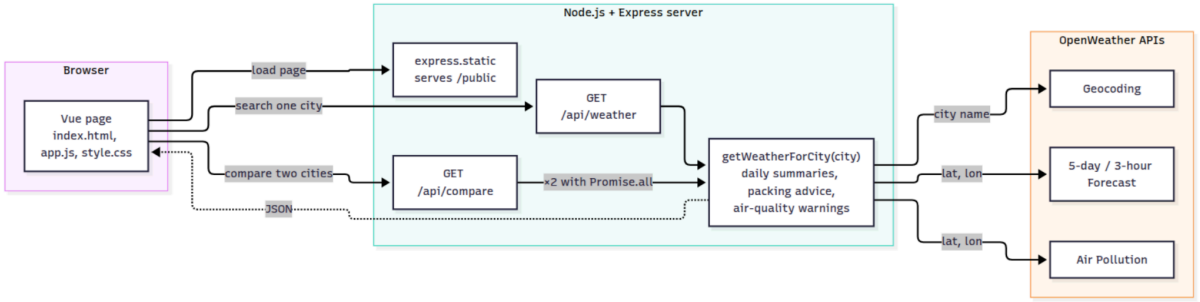


Figure 1: System architecture of the weather app. Solid arrows show requests and API data; the dotted arrow shows the JSON response.

Some main observations:

- **Shared weather logic:** The route handlers manage the HTTP requests and responses. `getWeatherForCity` does the shared lookup, using helpers such as `makeDailySummaries`, `makePackingAdvice`, and `makeAirQualitySummary` to prepare the data.
- **Parallel comparison:** `/api/compare` calls `getWeatherForCity` for both cities with `Promise.all`, so the same lookup logic is reused and the requests run in parallel.
- **Stateless server:** The server doesn't need to remember past searches. Each `GET` request is handled independently, with no need for sessions or storing data.
- **MVC paradigm**: In the client, the server's JSON is the model, the HTML template is the view, and the Vue instance is the controller.
- **Dependencies**: **Express** is the only package installed with npm. Node already includes fetch, and Vue loads from a CDN. Node also reads the API key from `.env` when the server starts.

3. Implementation

3.1 Server side

I found myself building the server in three iterations, each one answering a different question: first, can I get anything out of OpenWeather at all; then, what do I do with what comes back; and finally, small refinements after the previous version.

Iteration 1: the first version just called the `Geocoding` and `Forecast` APIs and passed the response straight through, with nothing processed and no error handling. This first version did two things:

- **Chain the two requests**. The handler reads the city from the query string, `awaits` Geocoding for its coordinates, then `awaits` Forecast with them. While it waits, Node's event loop can handle other requests.
- **Return the Forecast response as it comes**, which was useful to see exactly what I was working with. OpenWeather does not return "days" but a list of about forty entries, one every three hours, each with its own temperature, wind and rain values, so I had to group this into 3 days myself.

Iteration 2: This is where the logic of the app actually got build, on top of those responses:

- Daily summaries: The three-hour entries are grouped by **date**, and for each day the server computes **min,max** and **average** temperatures, the average **wind speed**, the total **rainfall** and whether rain is expected.
- Packing advice: The **Cold / Mild / Hot** thresholds form the specification are applied to each day's average, and the advice becomes "pack layers" if the days fall into different categories. The umbrella flag, on the other hand, is set if any day expects rain.
- Air quality: the Air Pollution API call was added. Each pollutant is compared with the upper limit of OpenWeather's "Good" band, and anything above it is reported with its band and a short health risk.
- Error handling: the route checks the city before doing anything, and every failure ends in an HTTP status code[2] and a message the client can show (Table 2).

Status	OpenWeather API
200 (OK)	Success: `city`, `today` ,`days[3]`, `packing`, `airQuality`
400 (Bad Request)	City missing or over 100 characters
404 (Not Found)	Geocoding returns no match
500 (Internal Server Error)	API key missing from `.env`
502 (Bad Gateway)	OpenWeather unreachable, returned an error, or returned too little data

Table 2: System architecture of the weather app. Status codes returned by `GET /api/weather`. The client only checks response.ok and shows the message.

Iteration 3: Once the core requirements were implemented in Iteration 2, I tested the app with more cities and at different times of day, surfacing some of the things that needed fixing:

- Days were grouped on UTC dates, which gave the wrong first day for cities far from Greenwich. Entries are now shifted by `city.timezone` first, and the current partial day is shown separately as "today".
- A failure in the Air Pollution request used to take the whole response down. It now has its own `try/catch`, so the forecast still renders if air quality is unavailable.
- The route handler had grown into a very long `app.get` . I moved the lookup into its own function, `getWeatherForCity`, which is what later made the comparison feature cheaper to add.

3.2 Sequence diagram of a search

Figure 2 shows what the final server does for one search. The browser sends the city to our `/api/weather`. The server then uses fetch with `async/await`: it first calls the `GeoCoding` API to turn the city into coordinates, then calls `Forecast` and `Air Pollution` with the same coordinates. The current code `awaits` Forecast before starting Air Pollution, although they could run in parallel. As mentioned before, Air Pollution is handled in a `try/catch`, so if it fails, the forecast can still be returned. Finally, after getting the data from OpenWeather, the server builds the daily summaries,

[2] For descriptions of HTTP status codes, see Wikipedia's https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

adds packing and air-quality information, and returns everything as one JSON object. The browser never sees the OpenWeather responses directly.

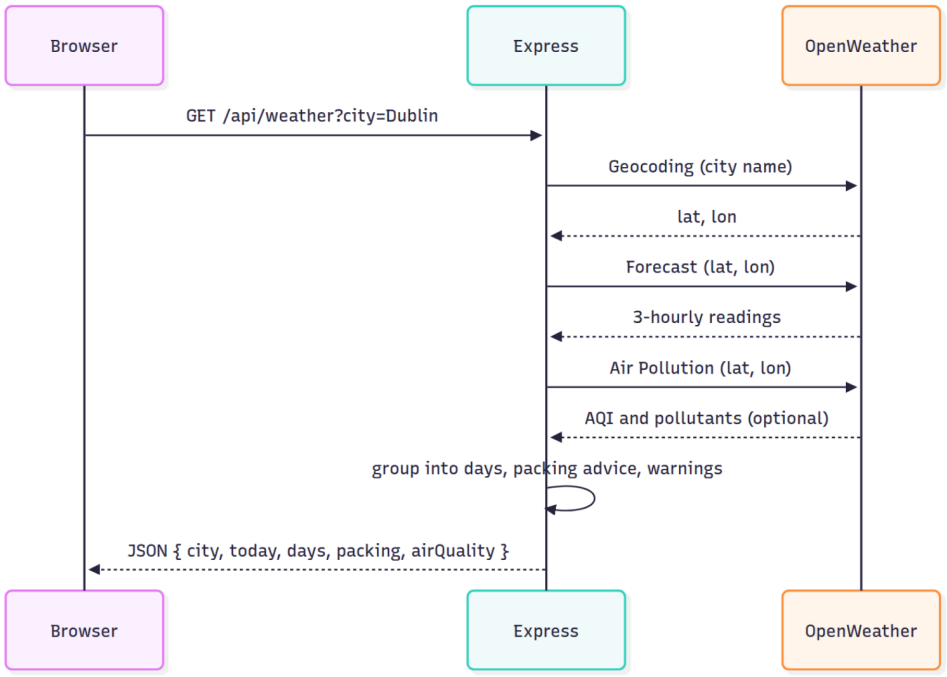


Figure 2: Server-side flow for a single weather search. Express gets the city's coordinates, fetches its forecast and air quality, then returns a daily summary as JSON

3.3 Client side

The client, on the other hand, was much more straightforward. Everything lives under `public/` : an `index.html` with the page structure and the Vue directives, an `app.js` with the Vue instance, and a separate `style.css` . I began with a skeleton that only had an input, a button and the raw JSON printed on screen, and then built the interface on top of that. The page has a small amount of state (the city typed in, whether a request is running, the result, and any error) and the HTML describes what to show for each of those. As in any reactive app, when the server replies the state changes and Vue updates the DOM for us (nothing needs to be updated by hand).

Within that, the HTML uses the usual Vue directives: `v-model` for the input, `v-if` to switch between the loading, error and result views, `v-for` for the forecast rows and pollutant warnings, and `:class` for the Cold / Mild / Hot colours. The JavaScript side is just the state, a method per form that sends the fetch request and stores the reply, and a few formatters.

For the layout, today's summary sits on the left and the forecast table, packing advice and air quality on the right, with an icon for each day's conditions

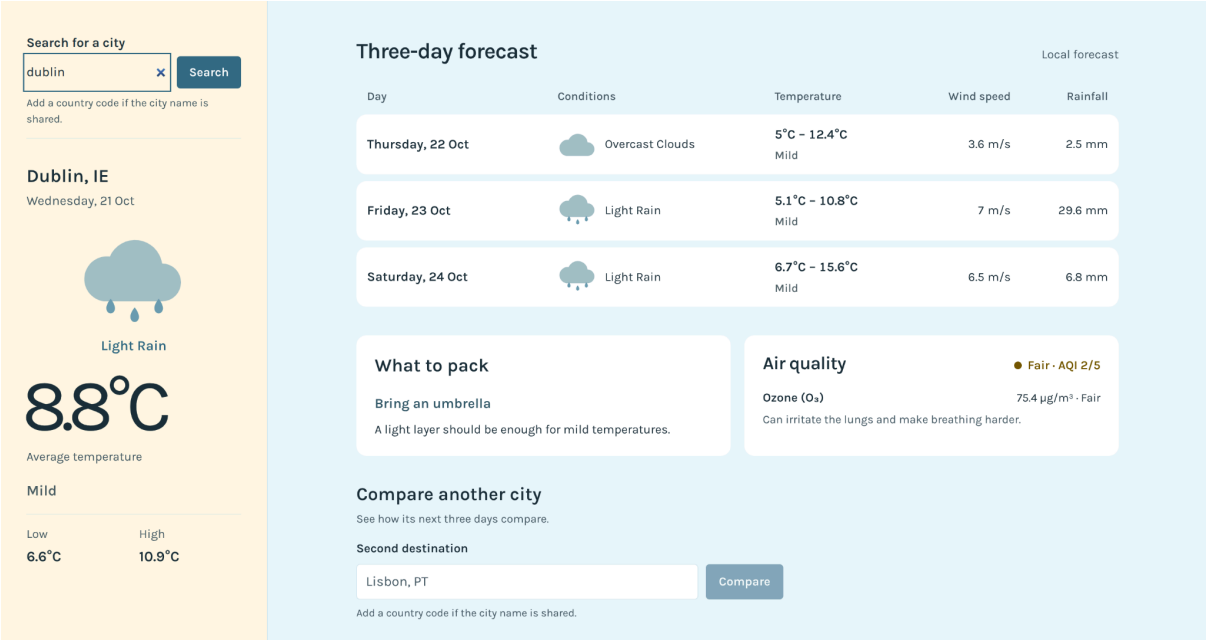


Figure 3: Result View for Dublin, showing current conditions, the three-day forecast, packing advice and air-quality information

4. Possible next steps

The obvious next step would be to deploy the server on **AWS**. I could also cache OpenWeather responses for a few minutes, as a single comparison currently makes **six API calls**: Geocoding, Forecast and Air Pollution for each city. Caching could avoid repeating those requests for recent searches. Deploying the app would also mean I could add **automated tests** for the routes and their error responses to help diagnose problems, a **/health** endpoint so AWS could check that the server is running, and **error logs** to help diagnose problems.

I would also improve the location selection for the coordinates: Geocoding returns several matches for an ambiguous city name, but our app currently uses the first one. A future version could let users choose the correct location, maybe from a drop-down. Additionally, the app currently only shows air-quality for the current reading; a future improvement could be to have it show predicted air quality for the upcoming forecast days, too, making it more useful when planning a trip.

Appendix A: Source Code

server.js

```
const express = require("express");
const path = require("node:path");

const app = express();
const port = process.env.PORT || 3000;
const host = process.env.HOST || "127.0.0.1";
const apiKey = process.env.OPENWEATHER_API_KEY;

// The browser calls this route with a city name.
app.get("/api/weather", async (req, res) => {
    const city = typeof req.query.city === "string" ?
req.query.city.trim() : "";

    if (!city || city.length > 100) {
        return res.status(400).json({
            error: "Enter a city name, optionally followed by a country
code."
        });
    }

    if (!apiKey) {
        return res.status(500).json({
            error: "The OpenWeather API key is missing. Add it to the
.env file and restart the server."
        });
    }

    try {
        return res.json(await getWeatherForCity(city));
    } catch (error) {
        return res.status(error.status || 502).json({ error:
error.message });
    }
});

// The browser sends both city names in the URL.
app.get("/api/compare", async (req, res) => {
    const cities = getRequestedCities(req.query.cities);

    if (cities.length !== 2 || cities.some((city) => !city || city.length
> 100)) {
        return res.status(400).json({
            error: "Enter two city names, for example Dublin and Lisbon."
        });
    }

    if (!apiKey) {
        return res.status(500).json({
```

```
            error: "The OpenWeather API key is missing. Add it to the
.env file and restart the server."
        });
    }

    try {
        // Both requests can run at the same time, so wait for them
together.
        const destinations = await Promise.all(
            cities.map((city) => getWeatherForCity(city))
        );

        return res.json({
            destinations,
            comparison: makeCityComparison(destinations)
        });
    } catch (error) {
        return res.status(error.status || 502).json({ error:
error.message });
    }
});

app.use(express.static(path.join(__dirname, "public")));

// Repeated values let a city with a country code keep its comma.
function getRequestedCities(value) {
    if (Array.isArray(value)) {
        return value.map((city) => String(city).trim());
    }
    if (typeof value === "string") {
        return value.split(",").map((city) => city.trim());
    }
    return [];
}

// Both routes use this to get the weather for one city.
async function getWeatherForCity(city) {
    // The forecast API needs coordinates, so look up the city first.
    const locationUrl = new
URL("https://api.openweathermap.org/geo/1.0/direct");
    locationUrl.search = new URLSearchParams({ q: city, limit: "1",
appid: apiKey });
    const locations = await getOpenWeatherJson(locationUrl);

    if (!locations.length) {
        const error = new Error("City not found. Try adding its country
code.");
        error.status = 404;
        throw error;
    }

    const location = locations[0];
```

```
    const forecastUrl = new
URL("https://api.openweathermap.org/data/2.5/forecast");
    forecastUrl.search = new URLSearchParams({
        lat: String(location.lat),
        lon: String(location.lon),
        units: "metric",
        appid: apiKey
    });
    const forecast = await getOpenWeatherJson(forecastUrl);
    // Make daily summaries, then keep today separate from the next three
days.
    const dailySummaries = makeDailySummaries(
        forecast.list,
        forecast.city.timezone || 0
    );
    const today = dailySummaries[0];
    const days = dailySummaries.slice(1, 4);

    if (!today || !days.length) {
        const error = new Error("OpenWeather did not return enough
forecast data.");
        error.status = 502;
        throw error;
    }

    // Air quality is separate, so the forecast can still load if this
fails.
    let airQuality;
    try {
        const airUrl = new
URL("https://api.openweathermap.org/data/2.5/air_pollution");
        airUrl.search = new URLSearchParams({
            lat: String(location.lat),
            lon: String(location.lon),
            appid: apiKey
        });
        const airData = await getOpenWeatherJson(airUrl);
        airQuality = makeAirQualitySummary(airData.list[0]);
    } catch {
        airQuality = {
            available: false,
            message: "Air-quality data is unavailable."
        };
    }

    return {
        city: {
            name: location.name,
            state: location.state || "",
            country: location.country || ""
        },
        today,
```

```
        days,
        packing: makePackingAdvice(days),
        airQuality,
        hasThreeDays: days.length === 3
    };
}

function formatCityName(city) {
    return [city.name, city.state, city.country].filter(Boolean).join(",
");
}

// Fewer rainy days wins. If they tie, the city with more Mild days wins.
function makeCityComparison(destinations) {
    const cityStats = destinations.map((destination) => {
        const days = destination.days;
        return {
            averageTemperature:
                days.reduce((total, day) => total +
day.averageTemperature, 0) / days.length,
            rainDays: days.filter((day) => day.rainExpected).length,
            mildDays: days.filter((day) => day.temperatureCategory ===
"Mild").length,
            aqi: destination.airQuality.available ?
destination.airQuality.aqi : null
        };
    });

    let betterWeatherIndex = null;
    if (cityStats[0].rainDays !== cityStats[1].rainDays) {
        betterWeatherIndex = cityStats[0].rainDays <
cityStats[1].rainDays ? 0 : 1;
    } else if (cityStats[0].mildDays !== cityStats[1].mildDays) {
        betterWeatherIndex = cityStats[0].mildDays >
cityStats[1].mildDays ? 0 : 1;
    }

    let worseAirIndex = null;
    if (
        cityStats[0].aqi !== null &&
        cityStats[1].aqi !== null &&
        cityStats[0].aqi !== cityStats[1].aqi
    ) {
        worseAirIndex = cityStats[0].aqi > cityStats[1].aqi ? 0 : 1;
    }

    const cityNames = destinations.map((destination) =>
formatCityName(destination.city));
    const weatherVerdict = betterWeatherIndex === null
        ? "Neither forecast has a clear weather advantage"
        : cityNames[betterWeatherIndex] + " has the better forecast";
```

```
    let umbrellaVerdict;
    if (cityStats[0].rainDays && cityStats[1].rainDays) {
        umbrellaVerdict = "Both cities may need an umbrella";
    } else if (cityStats[0].rainDays) {
        umbrellaVerdict = cityNames[0] + " may need an umbrella";
    } else if (cityStats[1].rainDays) {
        umbrellaVerdict = cityNames[1] + " may need an umbrella";
    } else {
        umbrellaVerdict = "Neither city is forecast to have rain";
    }

    let airVerdict;
    if (cityStats[0].aqi === null || cityStats[1].aqi === null) {
        airVerdict = "Air quality is unavailable for comparison";
    } else if (worseAirIndex === null) {
        airVerdict = "Both cities have the same AQI";
    } else {
        airVerdict = cityNames[worseAirIndex] + " has worse air quality";
    }

    return {
        metrics: cityStats,
        temperatureDifference: Math.abs(
            cityStats[0].averageTemperature -
cityStats[1].averageTemperature
        ),
        betterWeatherIndex,
        worseAirIndex,
        verdict: [weatherVerdict, umbrellaVerdict, airVerdict].join(". ")
+ "."
    };
}

async function getOpenWeatherJson(url) {
    let response;
    try {
        response = await fetch(url);
    } catch {
        const networkError = new Error("Could not connect to
OpenWeather.");
        networkError.status = 502;
        throw networkError;
    }

    // Convert OpenWeather's JSON response into a JavaScript object.
    const data = await response.json();
    if (!response.ok) {
        const error = new Error(
            "OpenWeather request failed with status " + response.status +
"."
        );
        error.status = response.status;
```

```
        throw error;
    }
    return data;
}

// The forecast has readings every three hours, so group them by local
date.
function makeDailySummaries(readings, timezoneOffset) {
    const daysByDate = new Map();

    for (const reading of readings || []) {
        const date = new Date((reading.dt + timezoneOffset) * 1000)
            .toISOString()
            .slice(0, 10);
        if (!daysByDate.has(date)) daysByDate.set(date, []);
        daysByDate.get(date).push(reading);
    }

    return Array.from(daysByDate, ([date, dayReadings]) => {
        const middleReading = dayReadings[Math.floor(dayReadings.length /
2)];
        const condition = middleReading.weather[0];
        const temperatures = dayReadings.map((reading) =>
reading.main.temp);
        const lows = dayReadings.map((reading) => reading.main.temp_min
?? reading.main.temp);
        const highs = dayReadings.map((reading) => reading.main.temp_max
?? reading.main.temp);
        const average = (values) =>
            values.reduce((sum, value) => sum + value, 0) /
values.length;
        const rainfall = dayReadings.reduce(
            (total, reading) => total + (reading.rain?.["3h"] || 0),
            0
        );
        const rainExpected = dayReadings.some((reading) => {
            const weather = reading.weather[0].main;
            return (
                (reading.rain?.["3h"] || 0) > 0 ||
                ["Rain", "Drizzle", "Thunderstorm"].includes(weather)
            );
        });
        const averageTemperature = average(temperatures);

        return {
            date,
            icon: condition.icon,
            conditionDescription: condition.description,
            minTemperature: Math.min(...lows),
            maxTemperature: Math.max(...highs),
            averageTemperature,
            temperatureCategory:
```

```
getTemperatureCategory(averageTemperature),
            averageWindSpeed: average(dayReadings.map((reading) =>
reading.wind.speed)),
            rainfall,
            rainExpected
        };
    });
}

function getTemperatureCategory(temperature) {
    if (temperature < 8) return "Cold";
    if (temperature <= 24) return "Mild";
    return "Hot";
}

function makePackingAdvice(days) {
    const categories = [...new Set(days.map((day) =>
day.temperatureCategory))];
    let temperatureAdvice;

    if (categories.length > 1) {
        temperatureAdvice =
            "Pack layers. The daily average moves between " +
categories.join(" and ") + ".";
    } else if (categories[0] === "Cold") {
        temperatureAdvice = "Pack a warm coat. The daily average is below
8°C.";
    } else if (categories[0] === "Hot") {
        temperatureAdvice = "Pack for warm weather. The daily average is
above 24°C.";
    } else {
        temperatureAdvice = "A light layer should be enough for mild
temperatures.";
    }

    const umbrellaNeeded = days.some((day) => day.rainExpected);
    return {
        umbrellaNeeded,
        umbrellaAdvice: umbrellaNeeded
            ? "Rain is forecast. Bring an umbrella."
            : "No rain is forecast in these three days.",
        temperatureAdvice
    };
}

function makeAirQualitySummary(reading) {
    const aqiLabels = ["", "Good", "Fair", "Moderate", "Poor", "Very
poor"];
    const pollutants = [
        {
            key: "so2",
            name: "Sulphur dioxide (SO₂)",
```

```
            limits: [20, 80, 250, 350],
            healthRisk: "Can irritate airways and may worsen asthma
symptoms."
        },
        {
            key: "no2",
            name: "Nitrogen dioxide (NO₂)",
            limits: [40, 70, 150, 200],
            healthRisk: "Can irritate airways and worsen respiratory
symptoms."
        },
        {
            key: "pm10",
            name: "Particulate matter (PM₁₀)",
            limits: [20, 50, 100, 200],
            healthRisk: "Can affect the lungs and aggravate heart or lung
conditions."
        },
        {
            key: "pm2_5",
            name: "Fine particulate matter (PM₂.₅)",
            limits: [10, 25, 50, 75],
            healthRisk: "Fine particles can enter the lungs and aggravate
heart or lung conditions."
        },
        {
            key: "o3",
            name: "Ozone (O₃)",
            limits: [60, 100, 140, 180],
            healthRisk: "Can irritate the lungs and make breathing
harder."
        },
        {
            key: "co",
            name: "Carbon monoxide (CO)",
            limits: [4400, 9400, 12400, 15400],
            healthRisk: "At high exposure, it can reduce the oxygen
carried through the body."
        }
    ];

    const warnings = [];
    for (const pollutant of pollutants) {
        const value = reading.components[pollutant.key];
        if (!Number.isFinite(value) || value < pollutant.limits[0])
continue;

        const limitIndex = pollutant.limits.findIndex((limit) => value <
limit);
        const categories = ["Fair", "Moderate", "Poor", "Very poor"];
        warnings.push({
            name: pollutant.name,
```

```
            value,
            category: limitIndex === -1 ? "Very poor" :
categories[Math.max(0, limitIndex - 1)],
            healthRisk: pollutant.healthRisk
        });
    }

    const aqi = reading.main.aqi;
    return {
        available: true,
        aqi,
        aqiLabel: aqiLabels[aqi] || "Unknown",
        warnings
    };
}

app.listen(port, host, () => {
    console.log("Weather planner running at http://localhost:" + port);
});

server-first-iteration.js
// This is a small version of server.js, just to show the request flow.
// Run it with: node --env-file=.env server-first-iteration.js

// Express lets us make routes for the browser to call.
const express = require("express");

const app = express();

// Node reads the key from .env when the server starts.
// The browser does not get the key.
const apiKey = process.env.OPENWEATHER_API_KEY;

// Use a different port so this can run alongside server.js.
const port = 3001;

// Try http://localhost:3001/api/weather?city=Dublin
// req is the request; res is how we send the reply.
// async lets this function wait for OpenWeather with await.
app.get("/api/weather", async (req, res) => {
    // The city is in the URL, after ?city=.
    const city = req.query.city;

    // Look up the city's coordinates first.
    const locationUrl = new
URL("https://api.openweathermap.org/geo/1.0/direct");

    // These are added after the ? in the URL.
    // q is the city, limit asks for one match, appid is the key.
    locationUrl.search = new URLSearchParams({
        q: city,
        limit: "1",
```

```
        appid: apiKey
    });

    // fetch sends the request and .json() reads the reply.
    const locationResponse = await fetch(locationUrl);
    const locations = await locationResponse.json();

    // Geocoding returns a list. We asked for one result, so use the
first one.
    const location = locations[0];

    // The forecast request needs the latitude and longitude.
    const forecastUrl = new
URL("https://api.openweathermap.org/data/2.5/forecast");

    // units=metric gives temperatures in Celsius.
    forecastUrl.search = new URLSearchParams({
        lat: location.lat,
        lon: location.lon,
        units: "metric",
        appid: apiKey
    });

    const forecastResponse = await fetch(forecastUrl);
    const forecastData = await forecastResponse.json();

    // The list has readings every three hours. Keep only what the page
needs.
    const readings = forecastData.list.map((reading) => ({
        // dt_txt is the time, main.temp is the temperature, and
description is the weather.
        time: reading.dt_txt,
        temperature: reading.main.temp,
        description: reading.weather[0].description
    }));

    // Send the shorter version back to the browser.
    res.json({
        city: location.name,
        readings
    });
});

// Start the server so it can receive requests.
app.listen(port, () => {
    console.log("Simple example server running at http://localhost:" +
port);
});
```

public/app.js

```
const app = Vue.createApp({
```

```
    data() {
        return {
            city: "",
            weatherData: null,
            error: "",
            loading: false,
            comparisonCity: "",
            comparisonData: null,
            comparisonError: "",
            comparing: false
        };
    },
    computed: {
        cityLabel() {
            return this.weatherData ?
this.formatCityName(this.weatherData.city) : "";
        }
    },
    methods: {
        async searchWeather() {
            const cityName = this.city.trim();
            if (!cityName || this.loading || this.comparing) {
                return;
            }

            this.loading = true;
            this.error = "";
            this.weatherData = null;
            this.comparisonData = null;
            this.comparisonError = "";

            try {
                // Ask our server for the weather in this city.
                const response = await fetch("/api/weather?city=" +
encodeURIComponent(cityName));
                this.weatherData = await this.readJsonResponse(response);
            } catch (requestError) {
                this.error = requestError.message;
            } finally {
                this.loading = false;
            }
        },
        formatCityName(city) {
            return [city.name, city.state,
city.country].filter(Boolean).join(", ");
        },
        async readJsonResponse(response) {
            // The server API should send JSON, not a web page.
            const contentType = response.headers.get("content-type") ||
"";
            if (!contentType.includes("application/json")) {
                throw new Error(
```

```
                    "The server returned a web page instead of JSON.
Start or restart the app with `npm start` from the project folder, then
reload this page."
                );
            }
            const responseData = await response.json();
            if (!response.ok) {
                throw new Error(responseData.error || "The weather
request failed. Please try again.");
            }
            return responseData;
        },
        async compareCities() {
            const secondCity = this.comparisonCity.trim();
            if (!this.weatherData || !secondCity || this.comparing ||
this.loading) {
                return;
            }

            this.comparing = true;
            this.comparisonError = "";
            this.comparisonData = null;

            // Add each city separately, so a comma in a country code
stays in the name.
            const cityParams = new URLSearchParams();
            cityParams.append("cities", this.cityLabel);
            cityParams.append("cities", secondCity);

            try {
                const response = await fetch("/api/compare?" +
cityParams.toString());
                this.comparisonData = await
this.readJsonResponse(response);
            } catch (requestError) {
                this.comparisonError = requestError.message;
            } finally {
                this.comparing = false;
            }
        },
        formatDate(dateString, short = false) {
            const date = new Date(dateString + "T12:00:00Z");
            return new Intl.DateTimeFormat("en-IE", {
                weekday: short ? "short" : "long",
                day: "numeric",
                month: "short",
                timeZone: "UTC"
            }).format(date);
        },
        weatherIcon(iconCode) {
            // Match OpenWeather's code to an icon in weather-icons.svg.
            const code = String(iconCode || "");
```

```
            const group = code.slice(0, 2);
            const night = code.endsWith("n");

            if (group === "01") return night ? "moon" : "sun";
            if (group === "02" || group === "03") {
                return night ? "partly-cloudy-night" :
"partly-cloudy-day";
            }
            if (group === "04") return "cloud";
            if (group === "09" || group === "10") return "rain";
            if (group === "11") return "storm";
            if (group === "13") return "snow";
            if (group === "50") return "mist";
            return "cloud";
        },
        formatNumber(value) {
            if (!Number.isFinite(value)) {
                return "—";
            }
            const cleanValue = Math.abs(value) < 0.05 ? 0 : value;
            return new Intl.NumberFormat("en-IE", {
                minimumFractionDigits: 0,
                maximumFractionDigits: 1
            }).format(cleanValue);
        },
        formatTemperature(value) {
            return this.formatNumber(value) + "°C";
        },
        formatRainfall(value) {
            if (!Number.isFinite(value)) {
                return "—";
            }
            if (value === 0) {
                return "0";
            }
            if (value < 0.05) {
                return "<0.1";
            }
            return this.formatNumber(value);
        }
    }
});

app.mount("#app");
```

public/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
```

```
initial-scale=1.0">
    <meta name="theme-color" content="#e8f5fb">
    <meta
      name="description"
      content="A simple three-day weather forecast, packing advice and
current air-quality information.">
    <title>Weather forecast</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link

href="https://fonts.googleapis.com/css2?family=Karla:wght@400;500;600;700
&display=swap"
      rel="stylesheet">
    <link rel="stylesheet" href="style.css">
    <script src="https://unpkg.com/vue@3/dist/vue.global.prod.js"
defer></script>
    <script src="app.js" defer></script>
  </head>
  <body>
    <div id="app">
      <div class="weather-shell">
        <aside class="side-panel">
          <form class="search-form" @submit.prevent="searchWeather"
:aria-busy="loading">
            <label for="city">Search for a city</label>
            <input
              id="city"
              v-model.trim="city"
              type="search"
              name="city"
              maxlength="100"
              autocomplete="off"
              placeholder="Dublin, IE"
              aria-describedby="city-hint"
              required>
            <button type="submit" :disabled="loading || comparing ||
!city.trim()">Search</button>
            <p id="city-hint" class="search-hint">
              Add a country code if the city name is shared.
            </p>
          </form>

          <section v-if="weatherData" class="today-summary"
aria-labelledby="city-name">
            <h2 id="city-name" class="city-name">{{ cityLabel }}</h2>
            <p class="day-label">{{ formatDate(weatherData.today.date)
}}</p>
            <div class="current-condition">
              <svg class="weather-icon" viewBox="0 0 128 128" width="144"
height="144" aria-hidden="true">
                <use :href="'/weather-icons.svg#' +
```

```
weatherIcon(weatherData.today.icon)"></use>
              </svg>
              <p>{{ weatherData.today.conditionDescription }}</p>
            </div>
            <p class="average-temperature">
              {{ formatTemperature(weatherData.today.averageTemperature)
}}
            </p>
            <p class="average-label">Average temperature</p>
            <p
              class="temperature-category"
              :class="'category-' +
weatherData.today.temperatureCategory.toLowerCase()"
            >
              {{ weatherData.today.temperatureCategory }}
            </p>
            <div class="high-low">
              <span>Low <strong>{{
formatTemperature(weatherData.today.minTemperature) }}</strong></span>
              <span>High <strong>{{
formatTemperature(weatherData.today.maxTemperature) }}</strong></span>
            </div>
          </section>
          <p v-else class="side-empty">
            Search a city to see its forecast and packing advice.
          </p>
        </aside>

        <main class="main-panel">
          <section v-if="loading" class="message-panel"
aria-live="polite">
            <p>Looking up the forecast...</p>
          </section>

          <section v-if="error" class="message-panel error-panel"
role="alert">
            <div>
              <h1>Couldn't load this forecast</h1>
              <p>{{ error }}</p>
            </div>
            <button class="retry-button" type="button"
@click="searchWeather" :disabled="loading">
              Try again
            </button>
          </section>

          <section v-if="!loading && !error && !weatherData"
class="message-panel empty-panel">
            <svg
              class="weather-icon empty-icon"
              viewBox="0 0 128 128"
              width="144"
```

```
              height="144"
              aria-hidden="true"
            >
              <use href="/weather-icons.svg#partly-cloudy-day"></use>
            </svg>
            <h1>Three-day forecast</h1>
            <p>Your temperature, wind, rainfall and air-quality details
will appear here.</p>
          </section>

          <div v-if="weatherData && !loading" class="weather-results">
            <section class="forecast-section"
aria-labelledby="forecast-title">
              <div class="section-heading">
                <h1 id="forecast-title">Three-day forecast</h1>
                <p>Local forecast</p>
              </div>
              <p v-if="!weatherData.hasThreeDays" class="notice">
                OpenWeather returned fewer than three forecast days. The
available days are shown below.
              </p>
              <div class="table-scroll" tabindex="0"
aria-label="Scrollable forecast table">
                <table class="forecast-table">
                  <thead>
                    <tr>
                      <th scope="col">Day</th>
                      <th scope="col">Conditions</th>
                      <th scope="col">Temperature</th>
                      <th scope="col">Wind speed</th>
                      <th scope="col">Rainfall</th>
                    </tr>
                  </thead>
                  <tbody>
                    <tr v-for="day in weatherData.days" :key="day.date">
                      <th scope="row">{{ formatDate(day.date) }}</th>
                      <td class="weather-condition"
data-label="Conditions">
                        <div class="condition-content">
                          <svg
                            class="weather-icon"
                            viewBox="0 0 128 128"
                            width="52"
                            height="52"
                            aria-hidden="true"
                          >
                            <use :href="'/weather-icons.svg#' +
weatherIcon(day.icon)"></use>
                          </svg>
                          <span>{{ day.conditionDescription }}</span>
                        </div>
                      </td>
```

```
                    <td data-label="Temperature">
                      <span class="temperature-range">
                        {{ formatTemperature(day.minTemperature) }} -
                        {{ formatTemperature(day.maxTemperature) }}
                      </span>
                      <span
                        class="table-category"
                        :class="'category-' +
day.temperatureCategory.toLowerCase()"
                      >
                        {{ day.temperatureCategory }}
                      </span>
                    </td>
                    <td data-label="Wind speed">{{
formatNumber(day.averageWindSpeed) }} m/s</td>
                    <td data-label="Rainfall">{{
formatRainfall(day.rainfall) }} mm</td>
                  </tr>
                </tbody>
              </table>
            </div>
          </section>

          <div class="details-grid">
            <section class="packing-section"
aria-labelledby="packing-title">
              <h2 id="packing-title">What to pack</h2>
              <p
                class="packing-advice"
                :class="weatherData.packing.umbrellaNeeded ?
'needs-umbrella' : 'no-umbrella'"
              >
                {{ weatherData.packing.umbrellaNeeded ? "Bring an
umbrella" : "No umbrella needed" }}
              </p>
              <p class="temperature-advice">{{
weatherData.packing.temperatureAdvice }}</p>
            </section>

            <section class="air-section" aria-labelledby="air-title">
              <div class="air-heading">
                <h2 id="air-title">Air quality</h2>
                <p
                  v-if="weatherData.airQuality.available"
                  class="air-status"
                  :class="'aqi-level-' + weatherData.airQuality.aqi"
                >
                  <span class="status-dot" aria-hidden="true"></span>
                  {{ weatherData.airQuality.aqiLabel }} · AQI {{
weatherData.airQuality.aqi }}/5
                </p>
              </div>
```

```
                <p v-if="!weatherData.airQuality.available"
class="air-message">
                  {{ weatherData.airQuality.message }}
                </p>
                <p v-else-if="weatherData.airQuality.warnings.length ===
0" class="air-message">
                  All measured pollutants are within OpenWeather's Good
range.
                </p>
                <ul v-else class="warning-list">
                  <li v-for="warning in weatherData.airQuality.warnings"
:key="warning.name">
                    <div class="warning-line">
                      <strong>{{ warning.name }}</strong>
                      <span>{{ formatNumber(warning.value) }} µg/m³ · {{
warning.category }}</span>
                    </div>
                    <p>{{ warning.healthRisk }}</p>
                  </li>
                </ul>
              </section>
            </div>

            <section class="comparison-section"
aria-labelledby="comparison-title">
              <div class="comparison-heading">
                <h2 id="comparison-title">Compare another city</h2>
                <p>See how its next three days compare.</p>
              </div>

              <form class="comparison-form"
@submit.prevent="compareCities" :aria-busy="comparing">
                <label for="comparison-city">Second destination</label>
                <div class="comparison-input-row">
                  <input
                    id="comparison-city"
                    v-model.trim="comparisonCity"
                    type="search"
                    name="comparisonCity"
                    maxlength="100"
                    autocomplete="off"
                    placeholder="Lisbon, PT"
                    required>
                  <button type="submit" :disabled="comparing ||
!comparisonCity.trim()">
                    {{ comparing ? "Comparing..." : "Compare" }}
                  </button>
                </div>
                <p class="search-hint">Add a country code if the city
name is shared.</p>
              </form>
```

```
              <p v-if="comparing" class="comparison-message"
aria-live="polite">
                Loading both forecasts...
              </p>
              <p v-if="comparisonError" class="comparison-message
comparison-error" role="alert">
                {{ comparisonError }}
              </p>

              <div v-if="comparisonData" class="comparison-results">
                <div class="comparison-verdict">
                  <h3>Weather verdict</h3>
                  <p aria-live="polite">{{
comparisonData.comparison.verdict }}</p>
                  <span>
                    Average temperatures differ by
                    {{
formatTemperature(comparisonData.comparison.temperatureDifference) }}.
                  </span>
                </div>

                <p
                  v-if="
                    !comparisonData.destinations[0].hasThreeDays ||
                    !comparisonData.destinations[1].hasThreeDays
                  "
                  class="notice comparison-notice"
                >
                  One city has fewer than three forecast days available.
The available days are shown.
                </p>

                <div class="comparison-grid">
                  <article
                    v-for="(destination, index) in
comparisonData.destinations"
                    :key="destination.city.name +
destination.city.country + index"
                    class="comparison-card"
                    :class="{
                      'comparison-card-winner':
                        index ===
comparisonData.comparison.betterWeatherIndex
                    }"
                  >
                    <div class="comparison-card-heading">
                      <h3>{{ formatCityName(destination.city) }}</h3>
                      <span
                        v-if="index ===
comparisonData.comparison.betterWeatherIndex"
                        class="winner-label"
```

```
                      >
                        Better forecast
                      </span>
                    </div>

                    <dl class="comparison-metrics">
                      <div>
                        <dt>Average temperature</dt>
                        <dd>
                          {{
formatTemperature(comparisonData.comparison.metrics[index].averageTempera
ture) }}
                        </dd>
                      </div>
                      <div>
                        <dt>Rainy days</dt>
                        <dd>
                          {{
comparisonData.comparison.metrics[index].rainDays }} /
                          {{ destination.days.length }}
                        </dd>
                      </div>
                      <div>
                        <dt>Air quality</dt>
                        <dd v-if="destination.airQuality.available">
                          {{ destination.airQuality.aqiLabel }} · AQI {{
destination.airQuality.aqi }}/5
                          <span
                            v-if="index ===
comparisonData.comparison.worseAirIndex"
                            class="worse-air-label"
                          >
                            Worse
                          </span>
                        </dd>
                        <dd v-else>Unavailable</dd>
                      </div>
                    </dl>

                    <ul class="comparison-days"
:aria-label="formatCityName(destination.city) + ' forecast'">
                      <li v-for="day in destination.days"
:key="day.date">
                        <span class="comparison-day-date">{{
formatDate(day.date, true) }}</span>
                        <svg
                          class="weather-icon"
                          viewBox="0 0 128 128"
                          width="40"
                          height="40"
                          aria-hidden="true"
                        >
```

```
                          <use :href="'/weather-icons.svg#' +
weatherIcon(day.icon)"></use>
                        </svg>
                        <span class="comparison-day-condition">{{
day.conditionDescription }}</span>
                        <span class="comparison-day-temperature">
                          {{ formatTemperature(day.minTemperature) }} -
                          {{ formatTemperature(day.maxTemperature) }}
                        </span>
                        <span class="comparison-day-details">
                          {{ formatNumber(day.averageWindSpeed) }} m/s
wind ·
                          {{ formatRainfall(day.rainfall) }} mm rain
                        </span>
                      </li>
                    </ul>
                  </article>
                </div>
              </div>
            </section>
          </div>
        </main>
      </div>
    </div>
  </body>
</html>
```

public/style.css

```
/* Main colours and default text */
:root {
  color-scheme: light;
  --page: #e2f2fa;
  --surface: #fff8e2;
  --main-surface: #e8f5fb;
  --tile: #ffffff;
  --ink: #1c303b;
  --muted: #435863;
  --line: #c9e2ec;
  --blue: #326b84;
  --sun-soft: #fff1c9;
  --amber: #745300;
  --red: #993e36;
  font-family: "Karla", "Trebuchet MS", sans-serif;
  color: var(--ink);
  background: var(--page);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
```

```
}

body {
  min-width: 320px;
  min-height: 100svh;
  margin: 0;
  background: var(--page);
  color: var(--ink);
  font-size: 18px;
  line-height: 1.5;
}

button,
input {
  font: inherit;
}

button {
  -webkit-tap-highlight-color: transparent;
}

.weather-shell {
  display: grid;
  grid-template-columns: minmax(290px, 350px) minmax(0, 1fr);
  width: 100%;
  min-height: 100svh;
  background: var(--main-surface);
}

/* City search and today's weather */
.side-panel {
  display: flex;
  flex-direction: column;
  padding: 42px 34px;
  border-right: 1px solid var(--line);
  background: var(--surface);
}

.search-form {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  align-items: center;
  gap: 9px 12px;
  padding-bottom: 14px;
  border-bottom: 1px solid var(--line);
}

.search-form label {
  grid-column: 1 / -1;
  margin-bottom: -4px;
  font-size: 17px;
  font-weight: 600;
```

```
}

.search-form input {
  width: 100%;
  min-width: 0;
  min-height: 42px;
  padding: 8px 0;
  border: 0;
  outline: 0;
  background: transparent;
  color: var(--ink);
  font-size: 17px;
}

.search-form input::placeholder {
  color: var(--muted);
  opacity: 1;
}

.search-form input:focus-visible {
  outline: 2px solid var(--blue);
  outline-offset: 3px;
}

.search-form button,
.retry-button {
  min-height: 42px;
  padding: 9px 16px;
  border: 0;
  border-radius: 5px;
  background: var(--blue);
  color: white;
  cursor: pointer;
  font-size: 16px;
  font-weight: 600;
}

.search-form button:hover:not(:disabled),
.retry-button:hover:not(:disabled) {
  background: #105a84;
}

.search-form button:disabled,
.retry-button:disabled {
  cursor: not-allowed;
  opacity: 0.55;
}

.search-hint {
  grid-column: 1 / -1;
  margin: 0;
  color: var(--muted);
```

```
  font-size: 14px;
}

:focus-visible {
  outline: 2px solid var(--blue);
  outline-offset: 3px;
}

.today-summary {
  margin-top: 34px;
}

.city-name {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
  line-height: 1.25;
  overflow-wrap: anywhere;
}

.day-label {
  margin: 5px 0 0;
  color: var(--muted);
  font-size: 17px;
}

.current-condition {
  margin: 24px auto 0;
  text-align: center;
}

.current-condition .weather-icon {
  display: block;
  width: 144px;
  height: 144px;
  object-fit: contain;
  margin: 0 auto;
}

.current-condition p {
  margin: -5px 0 0;
  color: var(--blue);
  font-size: 18px;
  font-weight: 600;
  text-transform: capitalize;
}

.average-temperature {
  margin: 18px 0 0 -5px;
  font-size: clamp(72px, 8vw, 96px);
  font-weight: 300;
  letter-spacing: -0.075em;
```

```
  line-height: 0.98;
  font-variant-numeric: tabular-nums;
}

.average-label {
  margin: 8px 0 0;
  color: var(--muted);
  font-size: 16px;
}

.temperature-category {
  margin: 18px 0 0;
  font-size: 19px;
  font-weight: 600;
}

.category-cold {
  color: var(--blue);
}

.category-mild {
  color: #4f6067;
}

.category-hot {
  color: var(--amber);
}

.high-low {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
  margin-top: 16px;
  padding-top: 14px;
  border-top: 1px solid var(--line);
  color: var(--muted);
  font-size: 16px;
}

.high-low span {
  display: flex;
  flex-direction: column;
  gap: 3px;
}

.high-low strong {
  color: var(--ink);
  font-size: 19px;
  font-weight: 600;
}

.side-empty {
```

```
  max-width: 25ch;
  margin: 30px 0 0;
  color: var(--muted);
  font-size: 17px;
}

/* Forecast and the information cards */
.main-panel {
  display: flex;
  align-items: center;
  min-width: 0;
  padding: 48px clamp(28px, 5vw, 72px);
  background: var(--main-surface);
}

.weather-results {
  width: 100%;
  max-width: 1000px;
  margin: 0 auto;
  align-self: flex-start;
}

.message-panel {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  justify-content: center;
  width: 100%;
  min-height: 390px;
}

.message-panel h1,
.forecast-section h1,
.air-section h2 {
  margin: 0;
  font-size: 29px;
  font-weight: 600;
  letter-spacing: -0.03em;
  line-height: 1.25;
}

.message-panel p {
  max-width: 42ch;
  margin: 9px 0 0;
  color: var(--muted);
  font-size: 18px;
}

.empty-panel .empty-icon {
  width: 152px;
  height: 152px;
  margin-bottom: 10px;
```

```
}

.error-panel {
  gap: 20px;
}

.error-panel h1 {
  color: var(--red);
}

.error-panel .retry-button {
  background: var(--red);
}

.section-heading {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 20px;
}

.section-heading > p {
  margin: 0;
  color: var(--muted);
  font-size: 16px;
}

.notice {
  margin: 0 0 14px;
  padding: 10px 13px;
  border-radius: 8px;
  background: var(--sun-soft);
  color: var(--amber);
  font-size: 15px;
}

.table-scroll {
  overflow-x: auto;
}

.forecast-table {
  width: 100%;
  min-width: 800px;
  border-collapse: separate;
  border-spacing: 0 9px;
  text-align: left;
}

.forecast-table th,
.forecast-table td {
  padding: 13px 14px;
```

```
  vertical-align: middle;
}

.forecast-table thead th {
  padding-top: 0;
  padding-bottom: 4px;
  color: var(--muted);
  font-size: 15px;
  font-weight: 500;
}

.forecast-table tbody th,
.forecast-table tbody td {
  background: var(--tile);
  font-size: 16px;
}

.forecast-table tbody th {
  width: 25%;
  border-radius: 13px 0 0 13px;
  font-size: 17px;
  font-weight: 600;
}

.forecast-table tbody tr td:last-child {
  border-radius: 0 13px 13px 0;
}

.forecast-table td:nth-child(4),
.forecast-table td:nth-child(5),
.forecast-table thead th:nth-child(4),
.forecast-table thead th:nth-child(5) {
  text-align: right;
  white-space: nowrap;
  font-variant-numeric: tabular-nums;
}

.condition-content {
  display: flex;
  align-items: center;
  gap: 10px;
}

.condition-content .weather-icon {
  display: block;
  flex: 0 0 auto;
}

.condition-content span {
  text-transform: capitalize;
}
```

```
.temperature-range {
  display: block;
  color: var(--ink);
  font-size: 17px;
  font-weight: 600;
  white-space: nowrap;
  font-variant-numeric: tabular-nums;
}

.table-category {
  display: block;
  margin-top: 3px;
  font-size: 15px;
  font-weight: 500;
}

.details-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 18px;
  margin-top: 28px;
}

/* The second city comparison */
.comparison-section {
  margin-top: 32px;
}

.comparison-heading {
  margin-bottom: 16px;
}

.comparison-heading h2 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
}

.comparison-heading p {
  margin: 3px 0 0;
  color: var(--muted);
  font-size: 15px;
}

.comparison-form {
  display: grid;
  gap: 8px;
  max-width: 560px;
}

.comparison-form label {
  font-size: 16px;
```

```
  font-weight: 600;
}

.comparison-input-row {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 10px;
}

.comparison-input-row input {
  width: 100%;
  min-width: 0;
  min-height: 44px;
  padding: 9px 12px;
  border: 1px solid var(--line);
  border-radius: 5px;
  background: var(--tile);
  color: var(--ink);
}

.comparison-input-row input::placeholder {
  color: var(--muted);
  opacity: 1;
}

.comparison-input-row input:focus-visible {
  outline: 2px solid var(--blue);
  outline-offset: 2px;
}

.comparison-input-row button {
  min-height: 44px;
  padding: 9px 17px;
  border: 0;
  border-radius: 5px;
  background: var(--blue);
  color: white;
  cursor: pointer;
  font-size: 16px;
  font-weight: 600;
}

.comparison-input-row button:hover:not(:disabled) {
  background: #105a84;
}

.comparison-input-row button:disabled {
  cursor: not-allowed;
  opacity: 0.55;
}

.comparison-form .search-hint {
```

```
  font-size: 14px;
}

.comparison-message {
  margin: 14px 0 0;
  color: var(--muted);
  font-size: 15px;
}

.comparison-error {
  color: var(--red);
}

.comparison-results {
  margin-top: 22px;
}

.comparison-verdict {
  margin-bottom: 18px;
}

.comparison-verdict h3 {
  margin: 0;
  font-size: 17px;
  font-weight: 600;
}

.comparison-verdict p {
  margin: 3px 0 0;
  font-size: 18px;
  font-weight: 600;
}

.comparison-verdict span {
  display: block;
  margin-top: 3px;
  color: var(--muted);
  font-size: 15px;
}

.comparison-notice {
  margin-top: 0;
}

.comparison-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 18px;
}

.comparison-card {
  min-width: 0;
```

```
  padding: 20px 22px;
  border-radius: 13px;
  background: var(--tile);
}

.comparison-card-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
}

.comparison-card-heading h3 {
  min-width: 0;
  margin: 0;
  font-size: 20px;
  font-weight: 600;
  overflow-wrap: anywhere;
}

.comparison-card-winner .comparison-card-heading h3 {
  color: var(--blue);
}

.winner-label {
  flex: 0 0 auto;
  color: var(--blue);
  font-size: 14px;
  font-weight: 600;
}

.comparison-metrics {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
  margin: 16px 0 12px;
}

.comparison-metrics div {
  min-width: 0;
}

.comparison-metrics dt {
  color: var(--muted);
  font-size: 13px;
}

.comparison-metrics dd {
  margin: 2px 0 0;
  font-size: 15px;
  font-weight: 600;
  font-variant-numeric: tabular-nums;
```

```
}

.worse-air-label {
  display: block;
  color: var(--amber);
  font-size: 13px;
  font-weight: 500;
}

.comparison-days {
  margin: 0;
  padding: 0;
  list-style: none;
}

.comparison-days li {
  display: grid;
  grid-template-columns: 48px 38px minmax(0, 1fr) max-content;
  align-items: center;
  gap: 1px 9px;
  padding: 10px 0;
  border-top: 1px solid var(--line);
}

.comparison-day-date {
  color: var(--muted);
  font-size: 14px;
  font-weight: 500;
}

.comparison-days .weather-icon {
  display: block;
  width: 38px;
  height: 38px;
}

.comparison-day-condition {
  min-width: 0;
  font-size: 15px;
  text-transform: capitalize;
}

.comparison-day-temperature {
  font-size: 15px;
  font-weight: 600;
  white-space: nowrap;
  font-variant-numeric: tabular-nums;
}

.comparison-day-details {
  grid-column: 3 / 5;
  color: var(--muted);
```

```
  font-size: 13px;
}

/* Put the cards in one column on smaller screens */
@media (max-width: 1200px) {
  .details-grid {
    grid-template-columns: 1fr;
  }

  .comparison-grid {
    grid-template-columns: 1fr;
  }
}

.packing-section,
.air-section {
  min-width: 0;
  padding: 22px 24px;
  border-radius: 13px;
  background: #fff;
}
.packing-section h2,
.air-section h2 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
}

.packing-advice {
  margin: 18px 0 0;
  font-size: 18px;
  font-weight: 600;
}

.needs-umbrella {
  color: #365b6b;
}

.no-umbrella {
  color: #336e4d;
}

.temperature-advice {
  margin: 9px 0 0;
  color: var(--ink);
  font-size: 16px;
}

.air-heading {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
```

```
  gap: 12px;
  margin-bottom: 18px;
}

.air-status {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  margin: 0;
  color: var(--aqi-color);
  font-size: 16px;
  font-weight: 600;
}

.aqi-level-1 {
  --aqi-color: #326344;
}

.aqi-level-2 {
  --aqi-color: #755900;
}

.aqi-level-3 {
  --aqi-color: #814817;
}

.aqi-level-4 {
  --aqi-color: #872f2a;
}

.aqi-level-5 {
  --aqi-color: #692b3c;
}

.status-dot {
  width: 10px;
  height: 10px;
  flex: 0 0 auto;
  border-radius: 50%;
  background: currentColor;
}

.air-message {
  margin: 0;
  color: var(--ink);
  font-size: 16px;
}

.warning-list {
  margin: 0;
  padding: 0;
  list-style: none;
```

```
}

.warning-list li + li {
  margin-top: 16px;
}

.warning-line {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
  font-size: 16px;
}

.warning-line strong {
  font-weight: 600;
}

.warning-line span {
  color: var(--ink);
  font-size: 15px;
  text-align: right;
}

.warning-list li p {
  margin: 5px 0 0;
  color: var(--muted);
  font-size: 15px;
}

/* Put the sidebar above the forecast */
@media (max-width: 1000px) {
  .weather-shell {
    display: block;
  }

  .side-panel {
    padding: 30px 28px;
    border-right: 0;
    border-bottom: 1px solid var(--line);
  }

  .today-summary {
    max-width: 460px;
  }

  .current-condition {
    margin-left: 0;
  }

  .main-panel {
    padding: 36px 28px;
```

```
  }
}

/* Smaller layout for phones */
@media (max-width: 650px) {
  .side-panel {
    padding: 24px 20px 28px;
  }

  .today-summary {
    margin-top: 26px;
  }

  .current-condition {
    margin-top: 16px;
  }

  .current-condition .weather-icon {
    width: 124px;
    height: 124px;
  }

  .average-temperature {
    margin-top: 12px;
    font-size: 76px;
  }

  .main-panel {
    min-height: 380px;
    padding: 30px 20px 36px;
  }

  .message-panel {
    min-height: 280px;
  }

  .details-grid {
    gap: 16px;
    margin-top: 24px;
  }

  .forecast-table {
    min-width: 860px;
  }

  .section-heading {
    align-items: flex-start;
  }

  .section-heading > p {
    text-align: right;
  }
```

```
.air-heading {
  flex-direction: column;
  align-items: flex-start;
  gap: 7px;
}

.warning-line {
  align-items: flex-start;
  flex-direction: column;
  gap: 2px;
}

.warning-line span {
  text-align: left;
}

.comparison-section {
  margin-top: 26px;
}

.comparison-input-row {
  grid-template-columns: minmax(0, 1fr);
}

.comparison-metrics {
  gap: 6px;
}

.comparison-metrics dt {
  font-size: 12px;
}

.comparison-metrics dd {
  font-size: 14px;
}

.comparison-days li {
  grid-template-columns: 42px 34px minmax(0, 1fr);
  gap: 1px 7px;
}

.comparison-days .weather-icon {
  width: 34px;
  height: 34px;
}

.comparison-day-date {
  grid-column: 1;
  grid-row: 1;
}
```

```
  .comparison-days .weather-icon {
    grid-column: 2;
    grid-row: 1 / 4;
  }

  .comparison-day-condition {
    grid-column: 3;
    grid-row: 1;
  }

  .comparison-day-temperature {
    grid-column: 3;
    grid-row: 2;
  }

  .comparison-day-details {
    grid-column: 3;
    grid-row: 3;
  }
}
```

public/weather-icons.svg

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="sun" viewBox="0 0 128 128">
    <g fill="none" stroke="#D8B247" stroke-linecap="round"
stroke-width="6">
      <path d="M64 10v14M64 104v14M10 64h14M104 64h14M26 26l10 10M92
92l10 10M102 26 92 36M36 92 26 102" />
    </g>
    <circle cx="64" cy="64" r="32" fill="#F2CF68" />
  </symbol>

  <symbol id="moon" viewBox="0 0 128 128">
    <path d="M91 19a49 49 0 1 0 18 89A51 51 0 0 1 91 19Z" fill="#E7C96F"
/>
  </symbol>

  <symbol id="partly-cloudy-day" viewBox="0 0 128 128">
    <g fill="none" stroke="#D8B247" stroke-linecap="round"
stroke-width="5">
      <path d="M77 12v10M108 44H98M99 21l-7 7M54 21l7 7" />
    </g>
    <circle cx="77" cy="44" r="25" fill="#F2CF68" />
    <path d="M31 105h65a21 21 0 0 0 0-42 31 31 0 0 0-60-8 25 25 0 0 0-5
50Z" fill="#A3BEC7" />
  </symbol>

  <symbol id="partly-cloudy-night" viewBox="0 0 128 128">
    <path d="M89 17a28 28 0 0 0 26 42A38 38 0 1 1 89 17Z" fill="#E7C96F"
/>
    <path d="M31 105h65a21 21 0 0 0 0-42 31 31 0 0 0-60-8 25 25 0 0 0-5
```

```
50Z" fill="#A3BEC7" />
  </symbol>

  <symbol id="cloud" viewBox="0 0 128 128">
    <path d="M29 101h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7
52Z" fill="#A3BEC7" />
  </symbol>

  <symbol id="rain" viewBox="0 0 128 128">
    <path d="M29 84h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7
52Z" fill="#A3BEC7" />
    <g fill="#6D9FB2">
      <path d="M42 92c0 8-10 8-10 0 0-5 5-11 5-11s5 6 5 11ZM70 102c0 8-10
8-10 0 0-5 5-11 5-11s5 6 5 11ZM98 92c0 8-10 8-10 0 0-5 5-11 5-11s5 6 5
11Z" />
    </g>
  </symbol>

  <symbol id="storm" viewBox="0 0 128 128">
    <path d="M29 80h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7
52Z" fill="#A3BEC7" />
    <path d="m66 70-18 28h16l-5 22 25-34H67l9-16Z" fill="#E7C96F" />
  </symbol>

  <symbol id="snow" viewBox="0 0 128 128">
    <path d="M29 80h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7
52Z" fill="#A3BEC7" />
    <g fill="#6D9FB2">
      <circle cx="39" cy="98" r="5" />
      <circle cx="65" cy="108" r="5" />
      <circle cx="91" cy="98" r="5" />
    </g>
  </symbol>

  <symbol id="mist" viewBox="0 0 128 128">
    <path d="M29 77h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7
52Z" fill="#A3BEC7" />
    <g fill="none" stroke="#6D9FB2" stroke-linecap="round"
stroke-width="6">
      <path d="M25 94h78M38 109h52" />
    </g>
  </symbol>
</svg>
```

Appendix B: Academic Integrity Declaration

I understand that this is an individual assessment and that collaboration is not permitted. I have not received any assistance with my work for this assessment. Where I have used the published work of others, I have indicated this with appropriate citation.

I have not and will not share any part of my work on this assessment, directly or indirectly, with any other student. I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at http://www.tcd.ie/calendar.

I understand that by returning this declaration with my work, I am agreeing with the above statement.

Name: Liliana Berenguer Jin
Date: 21st October 2026